School of Computer Science and Artificial Intelligence

Lab Assignment # 18.2

Program : B. Tech (CSE)

Specialization :AIML

Course Title : AI Assisted Coding

Course Code : 23CS002PC304

Semester : VI

Academic Session : 2025-2026

Name of Student : K.Srija

Enrollment No. : 2303A52189

Batch No. : 34

Date :31/03/26

Task 1: Air Quality API Integration Prompt:

```
Generate a Python script using requests to fetch real-time air quality data for a city.
Include error handling for invalid city, API key issues, and timeouts.
Display AQI, pollution level, and dominant pollutant.
```

Code:

```python
# Import necessary libraries
import requests
import json
from google.colab import userdata

# --- Configuration ---
API_KEY = userdata.get('AIzaSyAKt-FzR77SellVZqejRbHqGeOBQEZv0pA') # Retrieve API key from Colab secrets
GEOCODING_URL = "http://api.openweathermap.org/geo/1.0/direct"
AIR_POLLUTION_URL = "http://api.openweathermap.org/data/2.5/air_pollution"

def get_coordinates(city_name: str) -> tuple[float, float] | None:
    """Fetches latitude and longitude for a given city name."""
    params = {
        'q': city_name,
        'limit': 1,
        'appid': API_KEY
    }
    try:
        response = requests.get(GEOCODING_URL, params=params, timeout=5)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
        data = response.json()

        if not data:
```

```python
            print(f"Error: City '{city_name}' not found.")
            return None

        lat = data[0]['lat']
        lon = data[0]['lon']
        return lat, lon
    except requests.exceptions.HTTPError as e:
        if response.status_code == 401:
            print("Error: Invalid API key. Please check your OPENWEATHER_API_KEY secret.")
        else:
            print(f"HTTP error occurred while fetching coordinates: {e}")
    except requests.exceptions.ConnectionError:
        print("Error: A connection error occurred. Please check your internet connection.")
    except requests.exceptions.Timeout:
        print("Error: The request to get coordinates timed out.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred while fetching coordinates: {e}")
    except json.JSONDecodeError:
        print("Error: Could not decode JSON response from geocoding API.")
    return None

def get_air_quality_data(lat: float, lon: float) -> dict | None:
    """Fetches air quality data for given latitude and longitude."""
```

```python
        'lat': lat,
        'lon': lon,
        'appid': API_KEY
    }
    try:
        response = requests.get(AIR_POLLUTION_URL, params=params, timeout=5)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
        data = response.json()

        if not data or 'list' not in data or not data['list']:
            print("Error: No air quality data available for this location.")
            return None

        # OpenWeatherMap AQI (1-5 where 1=Good, 5=Very Poor)
        aqi_owm = data['list'][0]['main']['aqi']
        components = data['list'][0]['components']

        # Map OWM AQI to a more common scale/description
        aqi_description = {
            1: "Good",
            2: "Fair",
            3: "Moderate",
            4: "Poor",
```

```python
            1: "Good",
            2: "Fair",
            3: "Moderate",
            4: "Poor",
            5: "Very Poor"
        }.get(aqi_owm, "Unknown")

        # Determine dominant pollutant (simplified: highest concentration among common ones)
        # Note: A more accurate dominant pollutant calculation would compare against thresholds.
        # This is a heuristic for demonstration.
        common_pollutants = ['co', 'no', 'no2', 'o3', 'so2', 'pm2_5', 'pm10', 'nh3']
        dominant_pollutant = None
        max_concentration = -1

        for pollutant in common_pollutants:
            if pollutant in components and components[pollutant] > max_concentration:
                max_concentration = components[pollutant]
                dominant_pollutant = pollutant

        # If no common pollutant is found or has positive concentration
        if dominant_pollutant is None or max_concentration == -1:
            dominant_pollutant = "N/A (No dominant pollutant detected or common pollutants not present)"
```

```python
            dominant_pollutant = "N/A (No dominant pollutant detected or common pollutants not present)"

        return {
            'aqi': aqi_owm,
            'pollution_level': aqi_description,
            'dominant_pollutant': dominant_pollutant,
            'raw_components': components
        }

    except requests.exceptions.HTTPError as e:
        if response.status_code == 401:
            print("Error: Invalid API key. Please check your OPENWEATHER_API_KEY secret.")
        else:
            print(f"HTTP error occurred while fetching air quality data: {e}")
    except requests.exceptions.ConnectionError:
        print("Error: A connection error occurred. Please check your internet connection.")
    except requests.exceptions.Timeout:
        print("Error: The request for air quality data timed out.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred while fetching air quality data: {e}")
    except json.JSONDecodeError:
        print("Error: Could not decode JSON response from air quality API.")
    return None
```

```python
def display_aqi_results(city: str, aqi_data: dict):
    """Displays the air quality results in a user-friendly format."""
    print(f"\n--- Real-time Air Quality for {city.title()} ---")
    print(f"AQI (OpenWeatherMap scale 1-5): {aqi_data['aqi']}")
    print(f"Pollution Level: {aqi_data['pollution_level']}")
    print(f"Dominant Pollutant: {aqi_data['dominant_pollutant'].upper()}")
    # Optionally display all component concentrations
    # print("\nPollutant Concentrations (ug/m^3 or mg/m^3 for CO):")
    # for pollutant, value in aqi_data['raw_components'].items():
    #     print(f"  {pollutant.upper()}: {value}")

# --- Main execution ---
if __name__ == "__main__":
    if not API_KEY:
        print("Error: OPENWEATHER_API_KEY is not set. Please add your API key to Colab secrets.")
    else:
        city_input = input("Enter city name (e.g., London, New York): ")
        if not city_input:
            print("City name cannot be empty.")
        else:
            print(f"Fetching data for {city_input}...")
            coords = get_coordinates(city_input)
```

```python
if __name__ == "__main__":
    if not API_KEY:
        print("Error: OPENWEATHER_API_KEY is not set. Please add your API key to Colab secrets.")
    else:
        city_input = input("Enter city name (e.g., London, New York): ")
        if not city_input:
            print("City name cannot be empty.")
        else:
            print(f"Fetching data for {city_input}...")
            coords = get_coordinates(city_input)

            if coords:
                lat, lon = coords
                aqi_data = get_air_quality_data(lat, lon)

                if aqi_data:
                    display_aqi_results(city_input, aqi_data)
                else:
                    print("Could not retrieve air quality data.")
            else:
                print("Could not retrieve coordinates for the specified city.")
```

Task 2: Language Translation API Prompt:

```
Generate a Python program using requests to translate English text into two languages using

Take user input, handle errors, and display original and translated text.
```

Code:

```python
# Import necessary libraries
import requests
import json

# --- Configuration ---
TRANSLATE_API_URL = "https://libretranslate.com/translate"

def translate_text(text: str, source_lang: str, target_lang: str) -> str | None:
    """translates text using a public LibreTranslate instance."""
    # Create the payload as a Python dictionary
    payload_dict = {
        "q": text,
        "source": source_lang,
        "target": target_lang,
        "format": "text"
    }

    try:
        # Use the 'json' parameter for requests.post to let the library handle JSON serialization and headers
        response = requests.post(TRANSLATE_API_URL, json=payload_dict, timeout=10)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
        data = response.json()
        if 'translatedText' in data:
            return data['translatedText']
        else:
            print(f"Error: Translation failed for {target_lang}. API response: {data}")
            return None
    except requests.exceptions.HTTPError as e:
        print(f"HTTP error occurred during translation to {target_lang}: {e} (Status Code: {response.status_code})")
        if response.status_code == 429: # Too Many Requests
```

```python
    except requests.exceptions.HTTPError as e:
        print(f"HTTP error occurred during translation to {target_lang}: {e} (Status Code: {response.status_code})")
        if response.status_code == 429: # Too Many Requests
            print("You might be sending too many requests. Please wait a moment and try again.")
    except requests.exceptions.ConnectionError:
        print(f"Error: A connection error occurred while trying to translate to {target_lang}. Please check your Internet connection.")
    except requests.exceptions.Timeout:
        print(f"Error: The request to translate to {target_lang} timed out.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred while translating to {target_lang}: {e}")
    except json.JSONDecodeError:
        print(f"Error: Could not decode JSON response from translation API for {target_lang}.")
    return None

# --- Main execution ---
if __name__ == "__main__":
    english_text = input("Enter the English text you want to translate: ")

    if not english_text.strip():
        print("Error: Text to translate cannot be empty.")
    else:
        print(f"\nOriginal English Text: {english_text}")

        # Translate to Spanish
        spanish_translation = translate_text(english_text, "en", "es")
        if spanish_translation:
            print(f"Translated (Spanish): {spanish_translation}")
        else:
            print("Spanish translation failed.")

        # Translate to French
```

```python
        # Translate to French
        #rench_translation = translate_text(english_text, "en", "fr")
        if french_translation:
            print(f"Translated (French): {french_translation}")
        else:
            print("French translation failed.")
```

```
Enter the English text you want to translate: hello

Original English Text: hello
HTTP error occurred during translation to es: 400 Client Error: Bad Request for url: https://libretranslate.com/translate (Status Code: 400)
Spanish translation failed.
HTTP error occurred during translation to fr: 400 Client Error: Bad Request for url: https://libretranslate.com/translate (Status Code: 400)
French translation failed.
```

<u>Task 3: Book API</u> Prompt:

might be transient. If the problem persists, it might be necessary to

Generate a Python program using requests to fetch book details (title, author, publisher, year) using a free public API without an API key. Handle errors and invalid input.

Code:

```python
# Import necessary libraries
import requests
import json

# --- Configuration ---
OPEN_LIBRARY_API_URL = "http://openlibrary.org/search.json"

def get_book_details(title: str) -> list[dict] | None:
    """Fetches book details from Open Library API for a given title."""
    params = {
        'q': title,
        'limit': 5 # Fetch up to 5 results to show multiple options
    }
    try:
        response = requests.get(OPEN_LIBRARY_API_URL, params=params, timeout=30) # Increased timeout to 30 seconds
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
        data = response.json()

        if not data or 'docs' not in data or not data['docs']:
            print(f"No book details found for '{title}'.")
            return None

        books = []
        for book_data in data['docs']:
            book = {
                'title': book_data.get('title', 'N/A'),
                'author': ', '.join(book_data.get('author_name', ['N/A'])) if book_data.get('author_name') else 'N/A',
                'publisher': ', '.join(book_data.get('publisher', ['N/A'])) if book_data.get('publisher') else 'N/A',
                'first_publish_year': book_data.get('first_publish_year', 'N/A')
            }
            books.append(book)
        return books

    except requests.exceptions.HTTPError as e:
        print(f"HTTP error occurred while fetching book details: {e} (Status Code: {response.status_code})")
    except requests.exceptions.ConnectionError:
        print("Error: A connection error occurred. Please check your internet connection.")
    except requests.exceptions.Timeout:
        print("Error: The request to fetch book details timed out.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred while fetching book details: {e}")
    except json.JSONDecodeError:
        print("Error: Could not decode JSON response from Open Library API.")
    return None

def display_book_results(books: list[dict]):
    """Displays the book details in a user-friendly format."""
    print("\n--- Book Details Found ---")
    for i, book in enumerate(books):
        print(f"\nBook {i+1}:")
        print(f"  Title: {book['title']}")
        print(f"  Author(s): {book['author']}")
        print(f"  Publisher(s): {book['publisher']}")
        print(f"  First Publish Year: {book['first_publish_year']}")

# --- Main execution ---
if __name__ == "__main__":
    book_title = input("Enter the title of the book you want to search for: ")

    if not book_title.strip():
        print("Error: Book title cannot be empty.")
    else:
```

```python
# --- Main execution ---
if __name__ == "__main__":
    book_title = input("Enter the title of the book you want to search for: ")

    if not book_title.strip():
        print("Error: Book title cannot be empty.")
    else:
        print(f"Searching for book details for '{book_title}'...")
        book_details = get_book_details(book_title)

        if book_details:
            display_book_results(book_details)
        else:
            print("Could not retrieve book details.")
```

```
Enter the title of the book you want to search for: Harry Potter
Searching for book details for 'Harry Potter'...

--- Book Details Found ---

Book 1:
  Title: Harry Potter and the Philosopher's Stone
  Author(s): J. K. Rowling
  Publisher(s): N/A
  First Publish Year: 1997

Book 2:
  Title: Harry Potter and the Deathly Hallows
  Author(s): J. K. Rowling
  Publisher(s): N/A
  First Publish Year: 2007

Book 3:
  Title: Harry Potter and the Prisoner of Azkaban
  Author(s): J. K. Rowling
  Publisher(s): N/A
  First Publish Year: 1999

Book 4:
  Title: Harry Potter and the Goblet of Fire
  Author(s): J. K. Rowling
  Publisher(s): N/A
  First Publish Year: 2000

Book 5:
  Title: Harry Potter and the Order of the Phoenix
  Author(s): J. K. Rowling
  Publisher(s): N/A
  First Publish Year: 2003
```

Task 4: Job Listings Prompt:

Generate a Python program using requests to fetch job listings based on a keyword using a free API without an API key. Display job title, company, location, and handle errors with retry logic.

Code:

```python
# Import necessary libraries
import requests
import json
import time
from google.colab import userdata

# --- Configuration ---
# IMPORTANT: Replace these with actual API details from your chosen job API
# For example, if using Adzuna, you'd find your app_id and app_key.
# This example uses placeholders for a hypothetical API.
API_KEY = userdata.get('JOBS_API_KEY') # Retrieve API key from Colab secrets
JOB_SEARCH_API_URL = "https://api.example.com/jobs/v1/search" # REPLACE with actual API endpoint

MAX_RETRIES = 3
RETRY_DELAY_SECONDS = 5

def fetch_job_listings(keyword: str, retries: int = MAX_RETRIES) -> list[dict] | None:
    """Fetches job listings from a hypothetical job search API with retry logic."""
    if not API_KEY:
        print("Error: JOBS_API_KEY is not set. Please add your API key to Colab secrets.")
        return None

    # !!! IMPORTANT: Adjust these parameters according to the specific API you are using !!!
    params = {
        'app_id': 'YOUR_APP_ID', # If required by the API, e.g., for Adzuna
        'app_key': API_KEY,      # If the API key is passed as 'app_key'
        'what': keyword,         # Search keyword
        'where': 'us',           # Location (e.g., country code, city)
        'results_per_page': 10,  # Number of results per request
        'page': 1                # Page number
        # Add any other parameters specific to your chosen API
```

```python
for attempt in range(retries):
    try:
        print(f"Attempt {attempt + 1}/{MAX_RETRIES} to fetch job listings for '{keyword}'...")
        response = requests.get(JOB_SEARCH_API_URL, params=params, timeout=15)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
        data = response.json()

        # !!! IMPORTANT: Adjust this data parsing based on the actual API response structure !!!
        if data and 'results' in data and data['results']:
            return data['results']
        else:
            print(f"No job listings found for '{keyword}'.")
            return None

    except requests.exceptions.HTTPError as e:
        print(f"HTTP error occurred: {e} (Status Code: {response.status_code})")
        if response.status_code == 401:
            print("Error: Invalid API key or credentials. Please check your JOBS_API_KEY secret and any app_id/app_key settings.")
            return None # No point retrying for authentication errors
        elif response.status_code == 429: # Too many requests
            print("Rate limit hit. Retrying after delay...")
        else:
            print("Server error or other client-side issue. Retrying...")
    except requests.exceptions.ConnectionError:
        print("Connection error occurred. Retrying...")
    except requests.exceptions.Timeout:
        print("Request timed out. Retrying...")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred: {e}. Retrying...")
    except json.JSONDecodeError:
        print("Error: Could not decode JSON response from job search API. Retrying...")
```

```python
        if attempt < retries - 1:
            time.sleep(RETRY_DELAY_SECONDS)

    print(f"Failed to fetch job listings after {retries} attempts.")
    return None

def display_job_listings(job_listings: list[dict]):
    """Displays job listings in a user-friendly format."""
    print("\n--- Latest Job Listings ---")
    for i, job in enumerate(job_listings):
        # !!! IMPORTANT: Adjust these keys based on the actual API response structure !!!
        title = job.get('title', 'N/A')
        company = job.get('company', {}).get('display_name', 'N/A') if isinstance(job.get('company'), dict) else job.get('company_name', 'N/A')
        location = job.get('location', {}).get('display_name', 'N/A') if isinstance(job.get('location'), dict) else job.get('location_name', 'N/A')
        description_snippet = job.get('description', 'N/A').split('\n')[0][:100] + '...' # Get first line or snippet
        url = job.get('redirect_url', 'N/A')

        print(f"\nJob {i+1}:")
        print(f"  Title: {title}")
        print(f"  Company: {company}")
        print(f"  Location: {location}")
        print(f"  Description: {description_snippet}")
        print(f"  Apply URL: {url}")

# --- Main execution ---
if __name__ == "__main__":
    search_keyword = input("Enter job search keyword (e.g., 'Software Engineer', 'Data Scientist'): ")

    if not search_keyword.strip():
        print("Error: Job search keyword cannot be empty.")
    else:
```

```python
    else:
        print(f"Searching for '{search_keyword}' jobs...")
        listings = fetch_job_listings(search_keyword)

        if listings:
            display_job_listings(listings)
        else:
            print("No listings to display or an error occurred.")
```

Task 5: Stock Market

Prompt:

Generate a Python program using requests to fetch stock market data using a free API without an API key. Display price, high, low, and handle invalid symbols and errors.

Code:

```python
# Import necessary libraries
import requests
import json
from google.colab import userdata

# --- Configuration ---
API_KEY = userdata.get('ALPHA_VANTAGE_API_KEY') # Retrieve API key from Colab secrets
ALPHA_VANTAGE_URL = "https://www.alphavantage.co/query"

def get_stock_data(symbol: str) -> dict | None:
    """Fetches real-time stock data (price, high, low) for a given symbol."""
    if not API_KEY:
        print("Error: ALPHA_VANTAGE_API_KEY is not set. Please add your API key to Colab secrets.")
        return None

    params = {
        'function': 'GLOBAL_QUOTE',
        'symbol': symbol,
        'apikey': API_KEY
    }

    try:
        response = requests.get(ALPHA_VANTAGE_URL, params=params, timeout=10)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
        data = response.json()

        # Alpha Vantage returns 'Error Message' for invalid symbols or API key issues
        if 'Error Message' in data:
            print(f"Error fetching data for '{symbol}': {data['Error Message']}")
            if "Invalid API key" in data['Error Message'] or "call frequency" in data['Error Message']:
                print("Please check your API key or respect the API call frequency limit.")
```

```python
            return None
        if 'Note' in data: # Usually rate limit message for free tier
            print(f"API Note: {data['Note']}")
            print("You might have hit the API call frequency limit. Please wait and try again.")
            return None

        # Extract relevant global quote data
        global_quote = data.get('Global Quote', {})
        if not global_quote:
            print(f"No global quote data found for symbol '{symbol}'. It might be an invalid symbol.")
            return None

        # Keys from Alpha Vantage Global Quote
        # '05. price', '02. high', '03. low'
        price = float(global_quote.get('05. price', 0.0))
        high = float(global_quote.get('02. high', 0.0))
        low = float(global_quote.get('03. low', 0.0))

        return {
            'symbol': symbol.upper(),
            'price': price,
            'high': high,
            'low': low
        }

    except requests.exceptions.HTTPError as e:
        print(f"HTTP error occurred while fetching stock data: {e} (Status Code: {response.status_code})")
        if response.status_code == 401:
            print("Error: Invalid API key. Please check your ALPHA_VANTAGE_API_KEY secret.")
    except requests.exceptions.ConnectionError:
        print("Error: A connection error occurred. Please check your internet connection.")
```

```python
    except requests.exceptions.Timeout:
        print("Error: The request to fetch stock data timed out.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred while fetching stock data: {e}")
    except json.JSONDecodeError:
        print("Error: Could not decode JSON response from stock API.")
    except ValueError as e:
        print(f"Error parsing numerical data from API response: {e}")
    return None

def display_stock_data(stock_data: dict):
    """Displays stock data in a user-friendly format."""
    print(f"\n--- Stock Data for {stock_data['symbol']} ---")
    print(f"Current Price: $${stock_data['price']:.2f}")
    print(f"Today's High:   $${stock_data['high']:.2f}")
    print(f"Today's Low:    $${stock_data['low']:.2f}")

# --- Main execution ---
if __name__ == "__main__":
    stock_symbol = input("Enter a stock symbol (e.g., AAPL, MSFT, GOOGL): ").strip().upper()

    if not stock_symbol:
        print("Error: Stock symbol cannot be empty.")
    else:
        print(f"Fetching stock data for {stock_symbol}...")
        data = get_stock_data(stock_symbol)

        if data:
            display_stock_data(data)
        else:
            print(f"Could not retrieve stock data for {stock_symbol}.")
```